

Performance Testing of a Multithreaded Application in C#

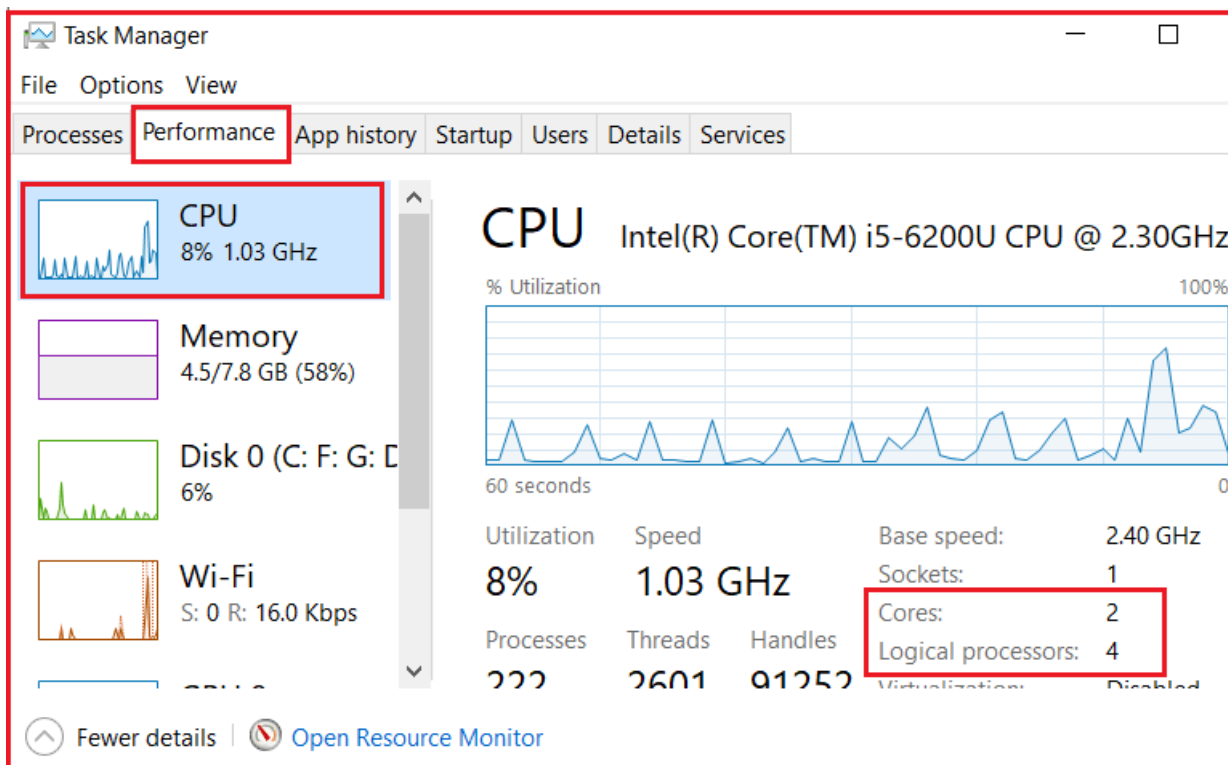
In this article, I am going to discuss the **Performance Testing of a multithreaded application in C#** with an example. Please read our previous article where we discussed **Deadlock in C#** with Examples. As part of this article, I will show you the performance implications of a multithreaded application when we run the application on a machine having a single core/processor versus a machine having a multi-core/processor.

How to find out how many processors you have on your machine?

You can find out how many processors you have on your machine in many different ways. Some of them are as follows:

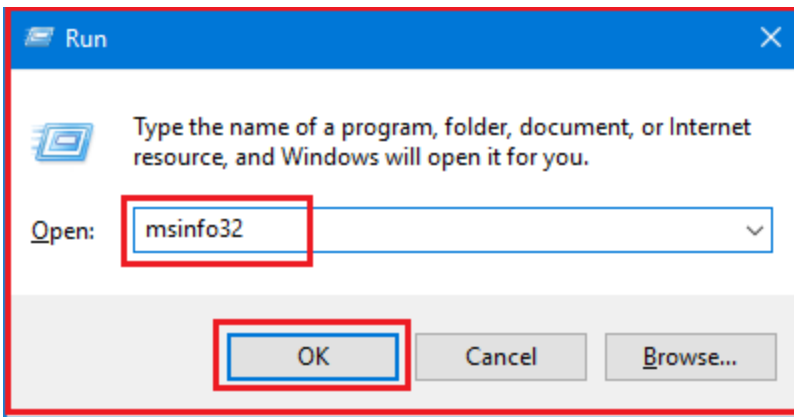
Way1: Using Task Manager

Right-click on the Taskbar and select the “**Task Manager**” option from the context menu. Then click on the “**Performance**” tab and select the “**CPU**” from the left side panel and then you will see the cores and Logical processors on the right side as shown in the below image.

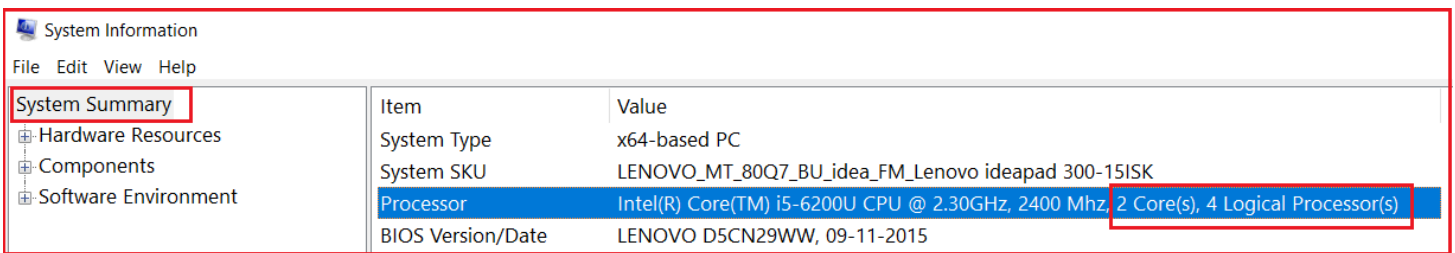


Way2: Using msinfo32 Command

Press the **Windows key + R** to open the **Run** command, then type **msinfo32** and click on the **OK** button as shown in the below image.



Once you click on the OK button, it will open up the System Information app. From that select the **Summary** option and scroll down until you find Processor. The details will tell you both how many cores and logical processors your CPU has as shown in the below image.



Way3: Using dot net code.

You can use the following code in any type of .net application to find out the total processors on the machine.

[Environment.ProcessorCount](#)

Way4: Using the command prompt

On the Windows command prompt write the following code and press enter

`echo %NUMBER_OF_PROCESSORS%`

Performance Testing With Multiple Processors:

If you have a machine with multiple processors, then multiple threads can execute your application code in parallel on different cores. For example, if your machine has two cores and there are two threads to execute your application code, then each thread is going to be run on an individual core. As a result, we will get better performance.

If you have two threads and if each thread takes 10 milliseconds to complete the execution, then on a machine with 2 processors, the total time taken is 10 milliseconds.

Performance Testing With Single Processor:

If you have a machine with a single processor, then multiple threads are going to execute one after the other. It is not possible for a single-core processor machine to execute multiple threads in parallel. The operating system switches between the threads so fast, it just gives us the illusion that the threads are running in parallel. On a single-core or processor machine, multiple threads can negatively affect the performance as there is overhead involved with context-switching.

If you have two threads and each thread takes 10 milliseconds to complete the execution, then on a machine with a single processor, the total time taken is 20 milliseconds plus thread context switching time if any.

Example: Using a Single Thread

```
using System;
using System.Diagnostics;
using System.Threading;
namespace MultithreadingPerformanceTesting
{
    class Program
    {
        public static void Main()
        {
            Stopwatch stopwatch = Stopwatch.StartNew();
            stopwatch = Stopwatch.StartNew();
            EvenNumbersSum();
            OddNumbersSum();
            stopwatch.Stop();
            Console.WriteLine($"Total time in milliseconds :
{stopwatch.ElapsedMilliseconds}");
            Console.ReadKey();
        }
        public static void EvenNumbersSum()
        {
            double Evensum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 0)
                {
                    Evensum = Evensum + count;
                }
            }
            Console.WriteLine($"Sum of even numbers = {Evensum}");
        }
        public static void OddNumbersSum()
        {
            double Oddsum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 1)
                {
                    Oddsum = Oddsum + count;
                }
            }
            Console.WriteLine($"Sum of odd numbers = {Oddsum}");
        }
    }
}
```

Output:

```
Sum of even numbers = 625000025000000
Sum of odd numbers = 625000000000000
Total time in milliseconds : 696
```

As you can see it takes approximately 696 milliseconds to complete the execution.

Example: Using Multiple Threads

Let's rewrite the previous example using multiple threads and compare the output.

```
using System;
using System.Diagnostics;
using System.Threading;
namespace MultithreadingPerformanceTesting
{
    class Program
    {
        public static void Main()
        {
            Stopwatch stopwatch = Stopwatch.StartNew();
            stopwatch = Stopwatch.StartNew();
            Thread thread1 = new Thread(EvenNumbersSum);
            Thread thread2 = new Thread(OddNumbersSum);
            thread1.Start();
            thread2.Start();
            thread1.Join();
            thread2.Join();
            stopwatch.Stop();
            Console.WriteLine($"Total time in milliseconds :
{stopwatch.ElapsedMilliseconds}");
            Console.ReadKey();
        }
        public static void EvenNumbersSum()
        {
            double Evensum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 0)
                {
                    Evensum = Evensum + count;
                }
            }
            Console.WriteLine($"Sum of even numbers = {Evensum}");
        }
        public static void OddNumbersSum()
        {
            double Oddsum = 0;
            for (int count = 0; count <= 50000000; count++)
            {
                if (count % 2 == 1)
```

```
        {  
            Oddsum = Oddsum + count;  
        }  
    }  
    Console.WriteLine($"Sum of odd numbers = {Oddsum}");  
}  
}  
}
```

Output:

```
Sum of even numbers = 625000025000000  
Sum of odd numbers = 625000000000000  
Total time in milliseconds : 508
```

As you can see, it takes approximately 508 milliseconds to complete the execution. In the next article, I am going to discuss [Thread Pooling in C#](#) with examples. Here, in this article, I try to show you the performance testing of a multithread application with an example. I hope you enjoy this article.